The grid-based approach to motion planning for a wheeled mobile robot was introduced by Barraquand and Latombe [8], and the grid-based approach to time-optimal motion planning for a robot arm with dynamic constraints was introduced in [24, 40, 39].

The GJK algorithm for collision detection was derived in [50]. Open-source collision-detection packages are implemented in the Open Motion Planning Library (OMPL) [181] and the Robot Operating System (ROS). An approach to approximating polyhedra with spheres for fast collision detection is described in [61].

The potential field approach to motion planning and real-time obstacle avoidance was first introduced by Khatib and is summarized in [73]. A search-based planner using a potential field to guide the search is described by Barraquand et al. [7]. The construction of navigation functions, potential functions with a unique local minimum, is described in a series of papers by Koditschek and Rimon [78, 76, 77, 152, 153].

Nonlinear optimization-based motion planning has been formulated in a number of publications, including the classic computer graphics paper by Witkin and Kass [194] using optimization to generate the motions of an animated jumping lamp; work on generating motion plans for dynamic nonprehensile manipulation [103]; Newton algorithms for optimal motions of mechanisms [88]; and more recent developments in short-burst sequential action control, which solves both the motion planning and feedback control problems [3, 187]. Path smoothing for mobile robot paths by subdivide and reconnect is described by Laumond et al. [82].

10.11 Exercises

Exercise 10.1 One path is **homotopic** to another if it can be continuously deformed into the other without moving the endpoints. In other words, it can be stretched and pulled like a rubber band, but it cannot be cut and pasted back together. For the C-space in Figure 10.2, draw a path from the start to the goal that is not homotopic to the one shown.

Exercise 10.2 Label the connected components in Figure 10.2. For each connected component, draw a picture of the robot for one configuration in the connected component.

Exercise 10.3 Assume that θ_2 joint angles in the range $[175°, 185°]$ result in

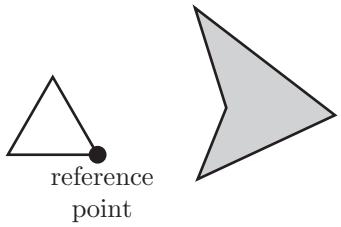

Figure 10.23: Exercise 10.4.

self-collision for the robot of Figure 10.2. Draw the new joint-limit C-obstacle on top of the existing C-obstacles and label the resulting connected components of $\mathcal{C}_{\text{free}}$. For each connected component, draw a picture of the robot for one configuration in the connected component.

Exercise 10.4 Draw the C-obstacle corresponding to the obstacle and translating planar robot in Figure 10.23.

Exercise 10.5 Write a program that accepts as input the coordinates of a polygonal robot (relative to a reference point on the robot) and the coordinates of a polygonal obstacle and produces as output a drawing of the corresponding C-space obstacle. In Mathematica, you may find the function `ConvexHull` useful. In MATLAB, try `convhull`.

Exercise 10.6 Calculating a square root can be computationally expensive. For a robot and an obstacle represented as collections of spheres (Section 10.2.2), provide a method for calculating the distance between the robot and obstacle that minimizes the use of square roots.

Exercise 10.7 Draw the visibility roadmap for the C-obstacles and q_{start} and q_{goal} in Figure 10.24. Indicate the shortest path.

Exercise 10.8 Not all edges of the visibility roadmap described in Section 10.3 are needed. Prove that an edge between two vertices of C-obstacles need not be included in the roadmap if either end of the edge does not hit the obstacle tangentially (i.e., it hits at a concave vertex). In other words, if the edge ends

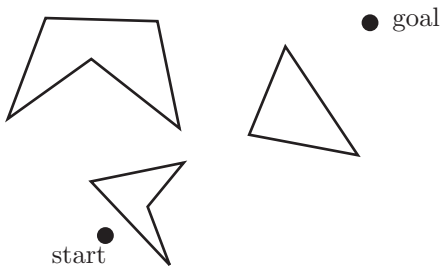

Figure 10.24: Planning problem for Exercise 10.7.

by "colliding" with an obstacle, it will never be used in a shortest path.

Exercise 10.9 Implement an A^* path planner for a point robot in a plane with obstacles. The planar region is a 100×100 area. The program generates a graph consisting of N nodes and E edges, where N and E are chosen by the user. After generating N randomly chosen nodes, the program should connect randomly chosen nodes by edges until E unique edges have been generated. The cost associated with each edge is the Euclidean distance between the nodes. Finally, the program should display the graph, search the graph using A^* for the shortest path between nodes 1 and N, and display the shortest path or indicate FAILURE if no path exists. The heuristic cost-to-go is the Euclidean distance to the goal.

Exercise 10.10 Modify the A^* planner in Exercise 10.9 to use a heuristic cost-to-go equal to ten times the distance to the goal node. Compare the running time with the original A^* when they are run on the same graphs. (You may need to use large graphs to notice any effect.) Are the solutions found with the new heuristic optimal?

Exercise 10.11 Modify the A^* algorithm from Exercise 10.9 to use Dijkstra's algorithm instead. Comment on the relative running times of A^* and Dijkstra's algorithm when each is run on the same graphs.

Exercise 10.12 Write a program that accepts the vertices of polygonal obstacles from a user, as well as the specification of a 2R robot arm, rooted at $(x, y) = (0, 0)$, with link lengths L_1 and L_2. Each link is simply a line segment.

Generate the C-space obstacles for the robot by sampling the two joint angles at k-degree intervals (e.g., $k = 5$) and checking for intersections between the line segments and the polygon. Plot the obstacles in the workspace, and in the C-space grid use a black square or dot at each configuration colliding with an obstacle. (Hint: At the core of this program is a subroutine to see whether two line segments intersect. If the segments' corresponding infinite lines intersect, you can check whether this intersection is within the line segments.)

Exercise 10.13 Write an A^* grid path planner for a 2R robot with obstacles and display on the C-space the paths you find. (See Exercise 10.12 and Figure 10.10.)

Exercise 10.14 Implement the grid-based path planner for a wheeled mobile robot (Algorithm 10.2), given the control discretization. Choose a simple method to represent obstacles and check for collisions. Your program should plot the obstacles and show the path that is found from the start to the goal.

Exercise 10.15 Write an RRT planner for a point robot moving in a plane with obstacles. Free space and obstacles are represented by a two-dimensional array, where each element corresponds to a grid cell in the two-dimensional space. The occurrence of a 1 in an element of the array means that there is an obstacle there, and a 0 indicates that the cell is in free space. Your program should plot the obstacles, the tree that is formed, and show the path that is found from the start to the goal.

Exercise 10.16 Do the same as for the previous exercise, except that obstacles are now represented by line segments. The line segments can be thought of as the boundaries of obstacles.

Exercise 10.17 Write a PRM planner to solve the same problem as in Exercise 10.15.

Exercise 10.18 Write a program to implement a virtual potential field for a 2R robot in an environment with point obstacles. The two links of the robot are line segments, and the user specifies the goal configuration of the robot, the start configuration of the robot, and the location of the point obstacles in the workspace. Put two control points on each link of the robot and transform the workspace potential forces to configuration space potential forces. In one workspace figure, draw an example environment consisting of a few point ob-

stacles and the robot at its start and goal configurations. In a second C-space figure, plot the potential function as a contour plot over (θ_1, θ_2), and overlay a planned path from a start configuration to a goal configuration. The robot uses the kinematic control law $\dot{q} = F(q)$.

See whether you can create a planning problem that results in convergence to an undesired local minimum for some initial arm configurations but succeeds in finding a path to the goal for other initial arm configurations.